



STUDENT PROJECT REPORT

Cricket Match Predictor

A command-line Python program that estimates a team's chance of winning from user-rated skills and match-day conditions

Student Name	Aditya Laxman Patil
Registration Number	26BCE10647
Project Type	Individual Python Programming Project (First Major Program)
Language / Tools	Python 3 (standard library only: datetime, json, pathlib, math)
Program File	main.py
Data Storage	Cricket_predictor.json (local history file)
Report Date	September 2026

PREPARED AS A PROJECT DOCUMENTATION REPORT

Table of Contents

1.	Cover Page	1
2.	Introduction	3
3.	Problem Statement	4
4.	Functional Requirements	5
5.	Non-Functional Requirements	6
6.	System Architecture	7
7.	Design Diagrams	8
8.	Design Decisions & Rationale	12
9.	Implementation Details	13
10.	Testing Approach	15
11.	Challenges Faced	16
12.	Learnings & Key Takeaways	17
13.	Future Enhancements	18
14.	References	19
A.	Program Execution Screenshots	20-21
	A.1 Program launch and match input	20
	A.2 Prediction output and history save	21

2. Introduction

Cricket fans often try to guess which side will win before a ball is even bowled. They weigh up which team is batting well at the moment, which team bowls better under the floodlights, and who looks sharper on the day. Usually this is done casually, in conversation, with no real structure behind it.

The **Cricket Match Predictor** is a small, text-based (command-line) Python program that turns this casual guessing into a simple, repeatable process. The user is asked a short series of questions — how strong is each team's batting, bowling and recent form, what is the pitch and weather like, how big are the boundaries, and who won the toss. The program then combines all of these answers using a straightforward points system and works out an estimated winning chance for each team, along with a short explanation of why.

This project was built as the author's first substantial Python program. It does not connect to the internet or use any live cricket data or statistics — every prediction is based purely on the numbers the user enters and a fixed set of rules built into the program. Every prediction made is also automatically saved to a small file on the computer, so a user can look back at earlier guesses later on.

In one line: the user rates both teams and describes the match conditions, and the program turns that into a friendly, numbers-based prediction.

3. Problem Statement

When following cricket, fans regularly compare two teams in their heads — "Team A has the stronger batting line-up, but Team B bowls better when it's cloudy." Doing this purely by feel has a few problems:

- It is **subjective** — two people looking at the same two teams can reach very different conclusions.
- It is **inconsistent** — the same person might weigh the same factors differently from one match to the next.
- It rarely accounts for **every factor together** — team skill, current form, pitch type, weather, ground size and the toss all matter, but it is hard to combine all of them in your head at once.
- There is **no record** kept of what was predicted and why, so there is no way to look back and see whether the reasoning made sense.

The goal of this project is to solve that with a lightweight, easy-to-use command-line tool: the user rates both teams and describes the match conditions once, and the program applies the same set of rules every time, giving a consistent score, a clear winning-chance percentage, and a short list of reasons behind the result — removing the guesswork and making the process repeatable.

4. Functional Requirements

Functional requirements describe *what* the program must do. These are the features that were implemented in `main.py`:

ID	Requirement	Description
FR1	Team name entry	Collect two team names from the user; both must be non-empty and the two names must be different from each other.
FR2	Skill ratings	Collect a batting, bowling and recent-form rating (on a scale of 1 to 5) for each of the two teams.
FR3	Pitch condition	Let the user choose the pitch type: batting-friendly, balanced, or bowling-friendly.
FR4	Weather condition	Let the user choose the weather: sunny, cloudy, or humid.
FR5	Boundary size	Let the user choose the ground's boundary size: small or large.
FR6	Toss details	Let the user record who won the toss (Team A, Team B, or no toss played) and what they chose to do (bat or field).
FR7	Base score calculation	Work out a base score for each team from its batting, bowling and form ratings.
FR8	Condition bonus points	Add extra points to whichever team's strengths best match the pitch, weather and boundary size.
FR9	Toss bonus points	Add a small bonus for winning the toss, plus an extra bonus if the toss winner's bat/field choice suits the conditions.
FR10	Win-percentage calculation	Convert the difference between the two teams' scores into an estimated winning percentage for each side.
FR11	Result display	Show the final scores, percentage chances, the reasons behind the bonus points, and the predicted winner on screen.
FR12	History logging	Automatically save every prediction — inputs and results — to a history file so it is not lost when the program closes.
FR13	Input validation	Catch invalid input (empty names, out-of-range ratings, unrecognised choices) and ask the user again instead of crashing.

5. Non-Functional Requirements

Non-functional requirements describe *how well* the program should behave, rather than which features it has:

Usability	Every question is asked in plain, friendly language, and the program keeps re-asking a question until a sensible answer is given, instead of stopping with an error.
Reliability	If the history file is missing, empty, or damaged in some way, the program quietly starts a fresh history list instead of crashing.
Portability	The program only uses tools that come built into Python (datetime, json, pathlib, math), so it can run on any computer that has Python 3 installed, with nothing extra to download.
Maintainability	The code is broken into small functions that each do one clear job (asking a question, building a team profile, calculating a score, saving a file), which makes it easier to read, test and improve later.
Performance	All the calculations are simple arithmetic and a single formula, so a prediction appears instantly with no noticeable waiting time.
Data persistence	Every prediction is written to a plain-text JSON file on disk, so results are kept permanently and can be opened and read even outside the program.

6. System Architecture

The Cricket Match Predictor is intentionally simple: it is a single Python file that runs entirely in a terminal window. There are no external servers, databases, or internet connections involved — every calculation happens on the user's own computer, and results are stored in a local file next to the program.

Even though it is one file, the code can be thought of as four logical layers, each handing information to the next:

1. **Input Layer** — small helper functions that ask the user questions and keep asking again if the answer is not valid (`team_name()` , `rating_instar()` , `get_option()`).
2. **Team Profile Layer** — gathers all three ratings for one team and bundles them into a single record (`team_n()`).
3. **Prediction Logic Layer** — the "brain" of the program. It compares both teams, adds bonus points for conditions and the toss, and turns the result into a win percentage (`get_conditions()` , `claci_prediction()`).
4. **Output & Storage Layer** — displays the result on screen and writes it to the JSON history file (`savemyprediction()`).

The `main()` function acts as the conductor: it calls each layer in the right order and passes information between them.

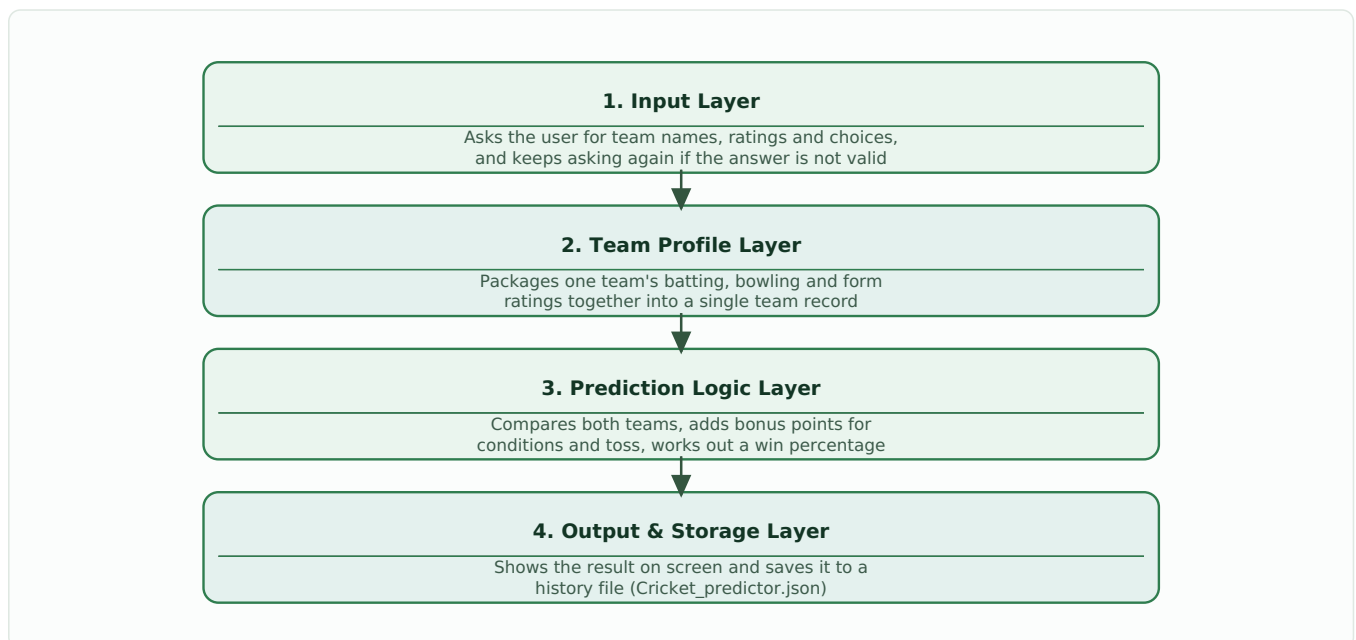


Figure 6.1 — The four logical layers of the program and how information flows between them.

7. Design Diagrams

This section presents the program's design visually, using simplified diagrams. Since the program is a small console application rather than a large multi-user system, the diagrams are kept intentionally simple and use plain-language labels.

7.1 Use Case Diagram

The single user of the program can perform six main actions. Two of these (rating the teams, and entering toss details) naturally happen as part of a bigger action, shown here as an `<<include>>` relationship.

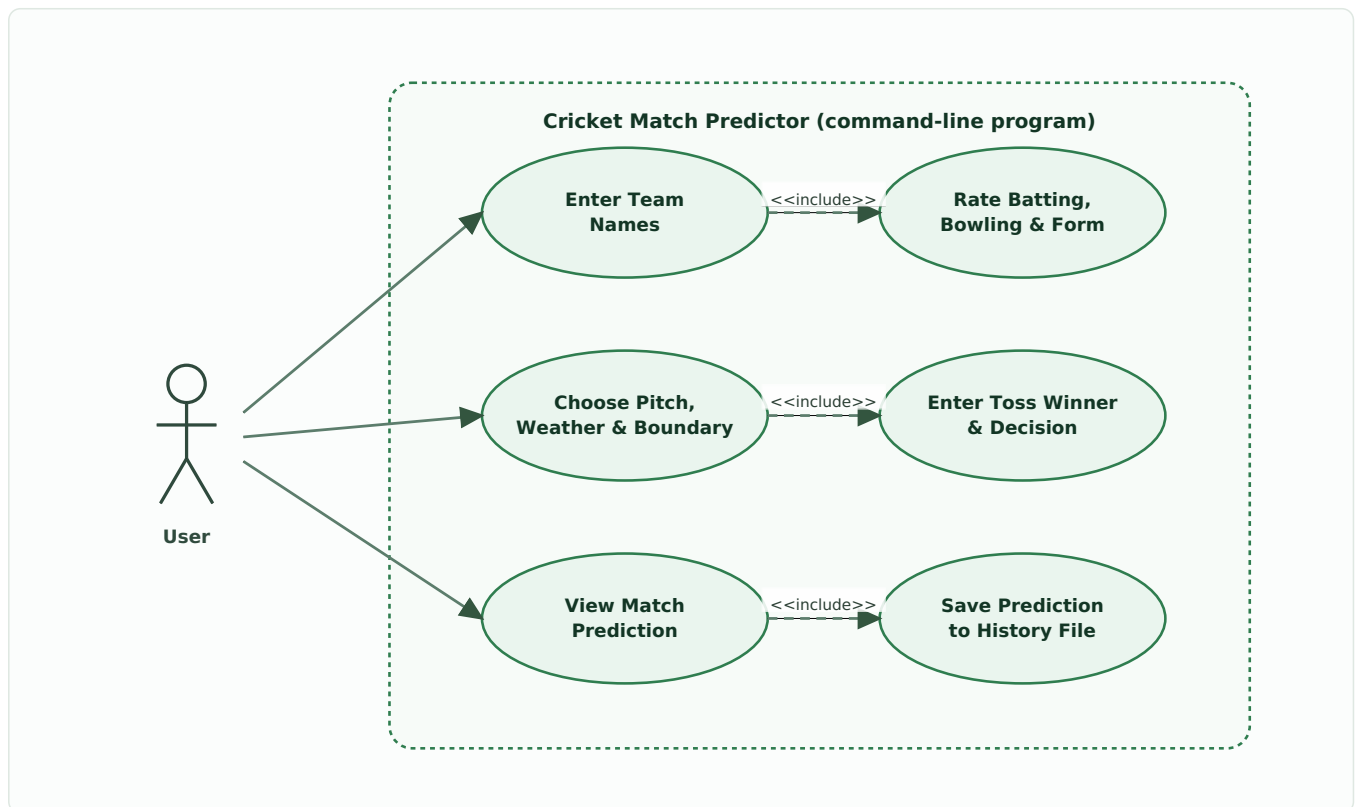


Figure 7.1 — Use Case Diagram: the user's interactions with the program.

7.2 Workflow Diagram

This flowchart shows the step-by-step journey of a single run of the program, from starting it to saving the final prediction.

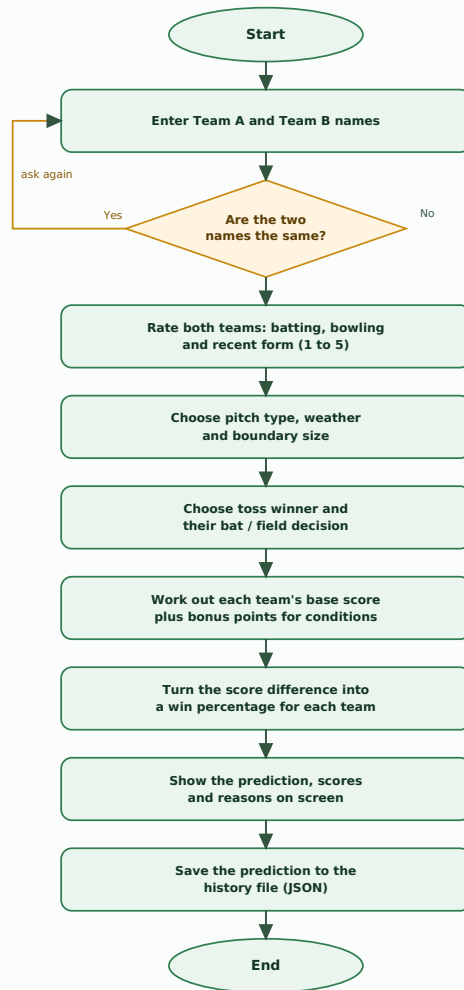


Figure 7.2 — Workflow Diagram: the full sequence of steps in one run of the program.

7.3 Sequence Diagram

This diagram shows the order in which the different parts of the program talk to each other while producing one prediction.

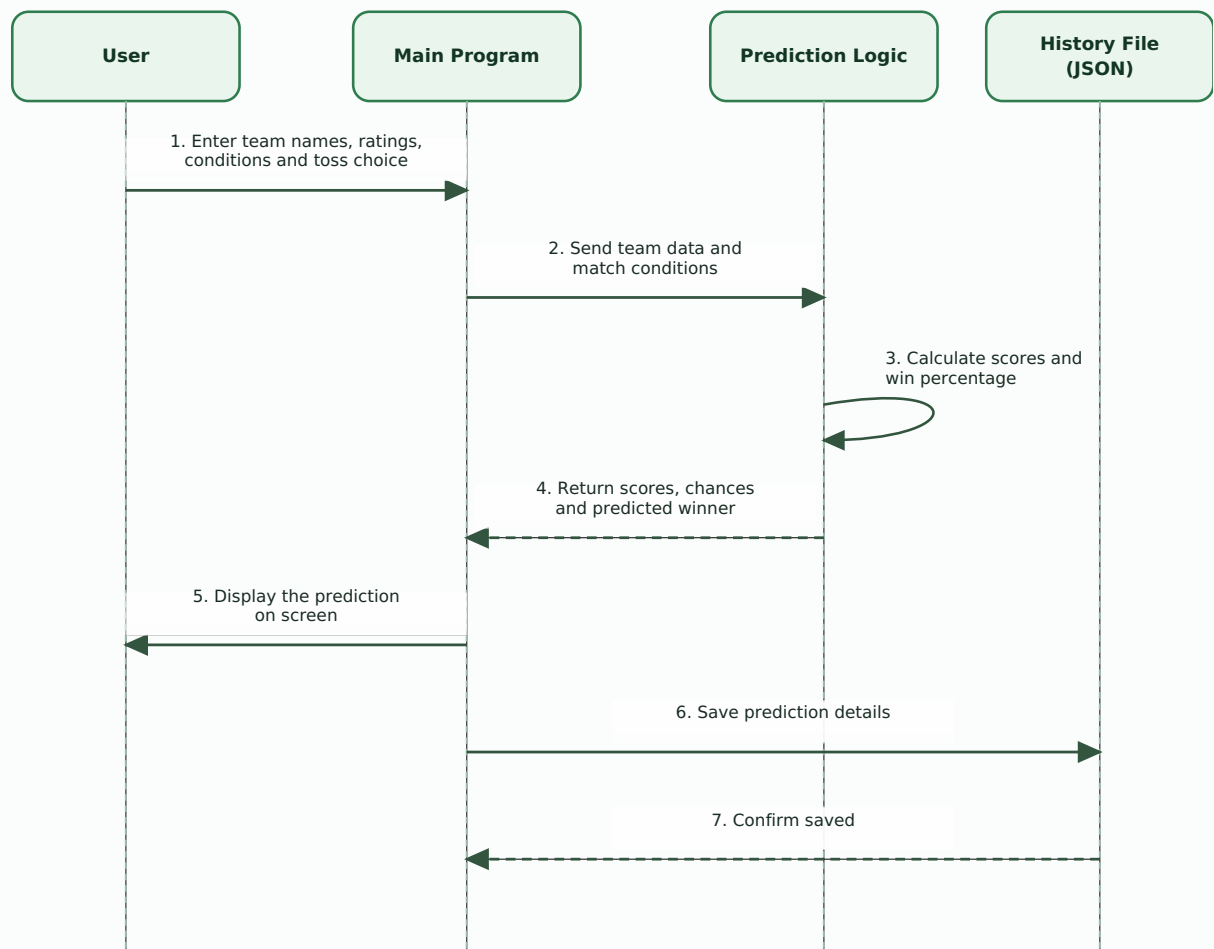


Figure 7.3 — Sequence Diagram: the messages exchanged while making and saving one prediction.

7.4 Class / Component Diagram

The program is written in a procedural style (functions rather than classes), so this diagram groups the functions into logical components and shows how they depend on one another — similar in spirit to a class diagram.

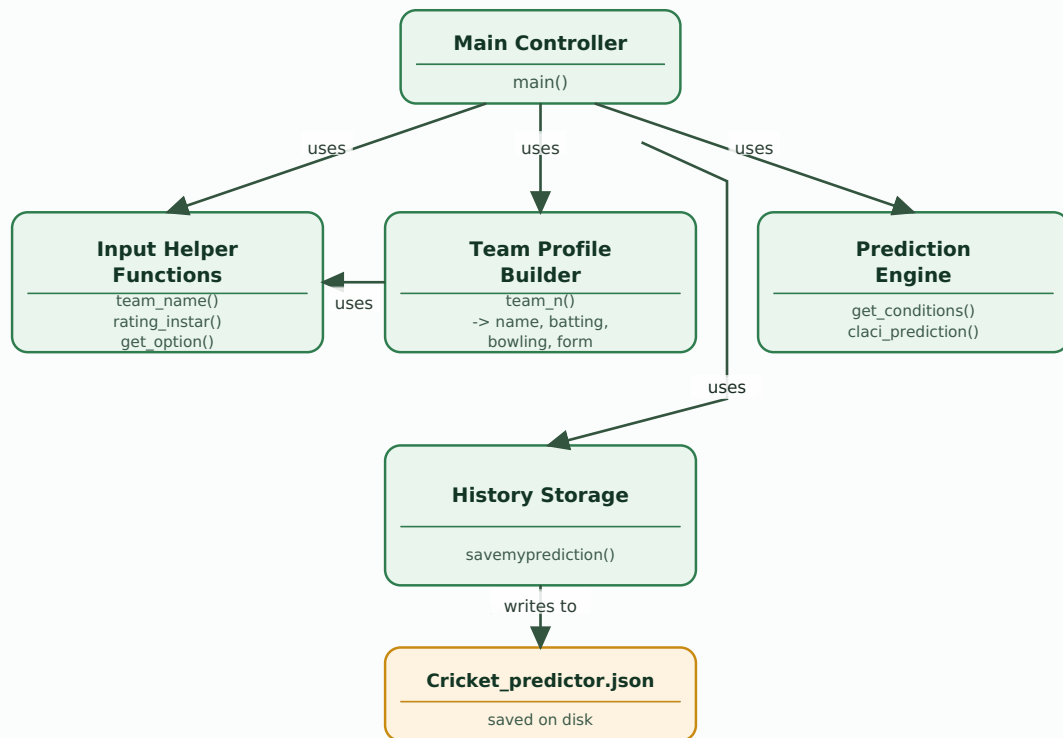
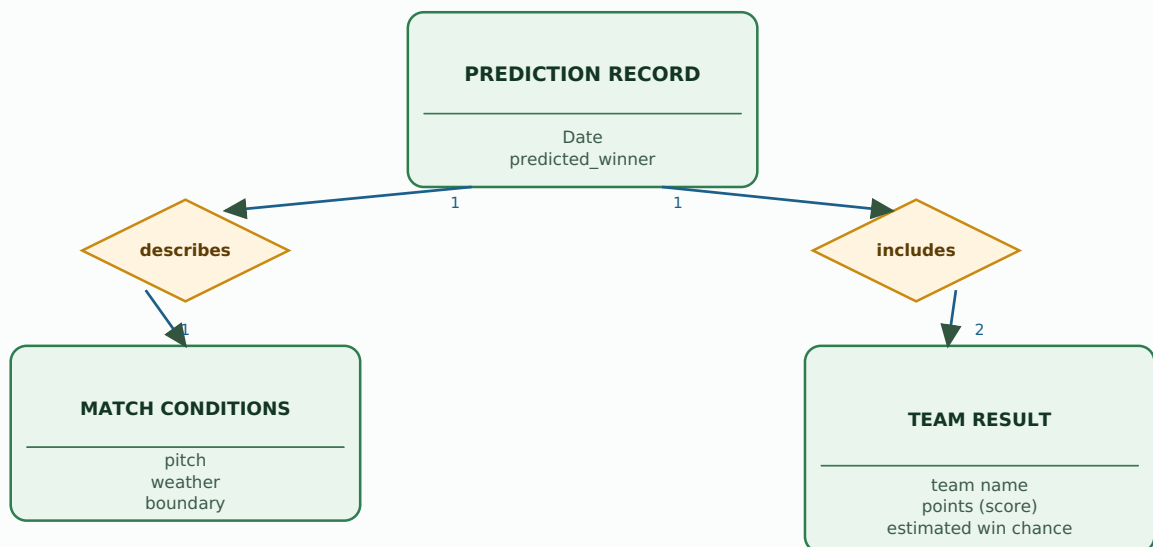


Figure 7.4 — Component Diagram: the program's functions grouped into logical building blocks.

7.5 ER Diagram (History File Storage)

The program stores data using a JSON file rather than a database, but the information saved still has a clear structure. This diagram shows that structure in entity-relationship form.



Each Prediction Record stores one set of match conditions and two Team Results (Team A and Team B).

Figure 7.5 — ER Diagram: the structure of a saved prediction record in Cricket_predictor.json.

8. Design Decisions & Rationale

A number of small design choices shaped how the program works. The table below lists the main ones and the thinking behind each.

Decision	Why it was made this way
Weighted scoring batting ×3, bowling ×3, form ×2	Batting and bowling are treated as the core, most important skills, so they carry the most weight. Recent form is still useful information but can be less reliable match to match, so it is given a slightly smaller weight.
Condition bonus points pitch, weather, boundary size	Real cricket knowledge says pitch type, weather and ground size favour either batting or bowling. Giving the stronger side in that area a few bonus points reflects this without making the maths complicated.
Toss bonus is conditional	Winning the toss alone is only a small advantage, so it earns one point. An extra point is only added if the toss winner's decision (bat or field) actually matches conditions that suit it — rewarding a "smart" choice rather than just winning a coin flip.
A smooth (logistic) formula for win %	Instead of a plain ratio, the score difference is passed through a mathematical S-shaped curve (the same idea used in many real prediction models). This keeps the percentage between 0% and 100% and avoids one team ever being shown as an impossible "0% chance".
JSON for saving history	JSON is a simple, human-readable text format that Python can read and write without installing anything extra. It avoids the need for a full database for what is a small amount of data.
Repeat-until-valid input loops	Typing mistakes are common. Instead of the program crashing on bad input, every question is asked inside a loop that politely asks again until a valid answer is given.
Everything in one file	As a first big program, keeping everything in a single, well-organised file made it easier to write, test and understand from start to finish.

9. Implementation Details

The program is written entirely in Python 3 and only uses modules that already come with Python, so nothing extra needs to be installed. Below is a plain-language summary of what each part of the code does.

9.1 Modules Used

Module	What it is used for
<code>datetime</code>	Records the date and time each prediction is made, so it can be stamped in the history file.
<code>json</code>	Reads and writes the history file in a structured, readable text format.
<code>pathlib</code>	Works out where the history file should be saved, right next to the program itself.
<code>math</code>	Provides the exponential function used in the win-percentage formula.

9.2 Key Functions

Function	Role
<code>team_name()</code>	Asks for a team name and keeps asking until something is typed in.
<code>rating_instar()</code>	Asks for a skill rating and only accepts whole numbers from 1 to 5.
<code>get_option()</code>	Asks the user to pick from a fixed list of choices (like pitch type) and only accepts one of those choices.
<code>team_n()</code>	Uses the three functions above to build one complete team profile: name, batting, bowling and form.
<code>get_conditions()</code>	Compares one skill between the two teams and gives bonus points to whichever team is stronger in it.
<code>claci_prediction()</code>	The core prediction engine — combines base scores, condition bonuses and toss bonuses, then works out the win percentage and predicted winner.
<code>savemyprediction()</code>	Adds the finished prediction to the history file, creating the file if it does not already exist.
<code>main()</code>	Runs the whole program from start to finish, calling the other functions in the right order.

9.3 How a Prediction is Calculated

Each team starts with a base score built from its ratings:

```
base score = (batting × 3) + (bowling × 3) + (form × 2)
```

Bonus points are then added on top of this base score, depending on the match conditions:

- **Pitch:** +4 points to the team with the better batting (batting-friendly pitch) or bowling (bowling-friendly pitch) rating.
- **Weather:** +2 points for the stronger batting side if sunny; +3 points for the stronger bowling side if cloudy; +2 points for the stronger bowling side if humid.
- **Boundary size:** +2 points for the stronger batting side on small boundaries, or the stronger bowling side on large boundaries.
- **Toss:** +1 point for winning the toss, and a further +1 point if the toss winner's bat/field decision matches conditions that suit it.

Once both teams' final scores are known, the difference between them is fed into a smooth mathematical formula that converts it into a percentage between 0% and 100% for each team, so the result always reads naturally (for example, "62% vs 38%") rather than a raw points total.

9.4 Data Storage Format

Every prediction is appended as one entry to a list stored in `Cricket_predictor.json`. Each entry records the date, both team names, the match conditions used, the final points for each team, the estimated win percentage for each team, and the predicted winner — so the full reasoning behind a past prediction can always be looked up again later.

10. Testing Approach

As a small command-line program, testing was done manually by running the program many times with different inputs and checking that the output made sense. No automated testing framework was used, but the following scenarios were specifically tried out:

Test Scenario	What was checked
Entering the same team name twice	The program should refuse and ask for a different second team name, rather than continuing.
Leaving a team name blank	The program should keep asking instead of accepting an empty name.
Typing letters instead of a rating number	The program should catch the error and ask again, instead of crashing.
Entering a rating outside 1-5 (like 0 or 9)	The program should reject it and ask again.
Choosing an option not in the list (like "windy" for weather)	The program should reject it and re-show the valid choices.
Both teams given identical ratings	The program should correctly report a tie in points rather than picking a winner.
Choosing "none" for the toss	The program should skip the toss bonus entirely and still produce a valid prediction.
Running the program for the first time (no history file yet)	The program should create a new history file rather than failing.
Running the program when the history file is damaged or unreadable	The program should start a fresh history list instead of crashing.
Extreme ratings (all 1s vs all 5s)	The stronger team should show a high winning percentage, but never exactly 100%.

Each of these was tested by hand, running the program in a terminal and observing the printed output, and by opening `Cricket_predictor.json` afterwards to confirm the saved entry matched what was shown on screen.

11. Challenges Faced

- **Handling bad input gracefully.** Early versions of the program would crash if a user typed a letter instead of a number for a rating. Wrapping these questions in a loop with a try/except block took some trial and error to get right.
- **Deciding on fair point values.** Choosing how many bonus points a pitch, weather, or boundary advantage should be worth (and how that compares to the base rating weights) needed some experimentation, so that conditions could influence the result without completely overriding the teams' actual skill ratings.
- **Turning a score difference into a percentage.** A simple ratio of the two scores could easily reach 0% or 100%, which does not feel realistic in a sport where surprises happen. Using a smooth mathematical curve (the logistic function) instead of a plain ratio took some research to understand and apply correctly.
- **Making the toss logic feel fair.** Deciding exactly which conditions count as "batting-friendly" versus "bowling-friendly" for the toss bonus (combining pitch, weather and boundary size together) needed careful thought so the bonus only applies when the toss winner's decision truly makes sense.
- **Keeping the history file safe.** Making sure the program does not crash the very first time it runs (when no history file exists yet), or when the file has somehow become empty or corrupted, required extra error handling around the file-reading step.
- **Readable console output.** Formatting the results so they are easy to read in a plain terminal window, without colours or graphics, took a few rounds of adjusting spacing and wording.

Known limitation: when the results are printed, the program currently prints the list of supporting reasons for Team B and the shared notes, but does not print Team A's individual list of reasons on screen (even though it is calculated correctly behind the scenes and reflected in the final score). This is a small display oversight rather than a calculation error, and is listed as a fix under Future Enhancements.

12. Learnings & Key Takeaways

As a first major program, this project was as much about learning good habits as it was about building a working tool. The main takeaways were:

- **Breaking a problem into small functions** makes it far easier to write, test, and fix code, compared to writing everything in one long block.
- **Validating user input early** saves a lot of trouble later — catching a bad answer immediately is much simpler than dealing with its effects further down the program.
- **Working with files** (reading and writing JSON with Python's built-in `json` module) is a practical, everyday skill that came directly from a real need in this project: keeping a history of results.
- **Using `try / except`** to handle things that might go wrong (bad input, a missing file) makes a program noticeably more reliable and pleasant to use.
- **Dictionaries are a natural way to model real things** — here, a team's name and three ratings fit neatly into one dictionary, which turned out to be easy to pass between functions.
- **A simple formula can go a long way.** Learning about the logistic (S-curve) function and seeing how it smooths out a raw score difference into a believable percentage was a useful first taste of applying basic mathematics inside a program.
- **Testing by hand, deliberately** (trying edge cases like a tie, or bad input) catches far more problems than only testing the "normal" path where everything is entered correctly.

13. Future Enhancements

The current version works well as a simple command-line tool, but there are several ways it could be extended if development continued:

Enhancement	Benefit
Fix the display so Team A's reasoning notes are also printed on screen	Gives the user the full, matching explanation for both teams, not just Team B.
Build a simple graphical or web interface	Makes the tool easier and more pleasant to use for people who are not comfortable with a terminal.
Player-level ratings instead of one overall team rating	Would allow more detailed, realistic predictions based on individual player form.
Connect to a real cricket statistics source	Would let ratings be suggested automatically from recent real-world results instead of being guessed by the user.
Show a chart of past predictions	Would let a user visually see how their predictions and estimated chances have changed over time.
Add more match conditions (day/night match, home ground advantage)	Would make the bonus-point system reflect a few more real factors that affect cricket outcomes.
Add automated tests	Would make it much faster to check nothing has broken whenever the code is changed in future.
Let the user export a prediction as a shareable text or image file	Would make it easy to share a prediction with friends outside the program itself.

14. References

- Python Software Foundation — *Python 3 Official Documentation* (`datetime`, `json`, `pathlib` and `math` modules), docs.python.org.
- Python Software Foundation — *The Python Tutorial*, docs.python.org/3/tutorial/, used as a general reference for Python syntax and standard-library usage while writing the program.
- General reference material on the *logistic (sigmoid) function*, used as the basis for converting a raw score difference into a bounded win-percentage estimate.
- General cricket knowledge (pitch types, weather effects on swing and spin, boundary size, and the importance of winning the toss) used informally to decide which bonus points made sense — not drawn from any single external source.
- `main.py` — the program itself, written by the project author, which this report documents.

This report was prepared as accompanying documentation for the Cricket Match Predictor student project and describes the program as implemented in `main.py`.

Appendix A - Program Execution Screenshots

```
Windows PowerShell
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

PS C:\Users\AVITA\OneDrive\Desktop\py> python main.py
*****
      CRICKET MATCH PREDICTOR
*****
A student project which predicts on probability if your team can win
It is only a rough guess, please don't take it to heart and loose hope in your favorites
Enter your team name: India
Enter your opponent team: Pakistan

Rating for India
Rate based on your Judgement, on the scale of 1 to 5
India batting rating (1-5): 4
India bowling rating (1-5): 3
India recent Form rating (1-5): 4

Rating for Pakistan
Rate based on your Judgement, on the scale of 1 to 5
Pakistan batting rating (1-5): 4
Pakistan bowling rating (1-5): 5
Pakistan recent Form rating (1-5): 3

Match conditions
Pitch batting/balanced/bowling.batting
weather sunny/cloudy/humid.cloudy
boundary size small/large.small

Toss team : A = India, B=Pakistan
Who won the toss a/b/none.a
Toss winner choose to bat/field.field

*****
      PREDICTION
*****
```

Figure A.1 - Program launch, team ratings, and match-condition input.

Appendix A - Program Execution Screenshots (continued)

```
Windows PowerShell
Rate based on your Judgement, on the scale of 1 to 5
Pakistan batting rating (1-5): 4
Pakistan bowling rating (1-5): 5
Pakistan recent Form rating (1-5): 3

Match conditions
Pitch batting/balanced/bowling.batting
weather sunny/cloudy/humid.cloudy
boundary size small/large.small

Toss team : A = India, B=Pakistan
who won the toss a/b/none.a
Toss winner choose to bat/field.field

*****
PREDICTION
*****

India: 31 points | estimated chance : 39.7%

Pakistan: 36 points | estimated chance : 60.3%
+3: stronger Bowling in Cloudy weather
  No Batting on friendly pitch points: both teams have the same rating
  No Batting on a ground with Small boundaries points: both teams have the same rating
  India won the toss and chose to field

My prediction: Pakistan is more likely to win based on your input
Remember: this is a points based estimation and not a guaranteed result
Prediction saved in Cricket_predictor.json
PS C:\Users\AVITA\OneDrive\Desktop\py>
```

Figure A.2 - Prediction points, estimated chances, and successful history save.